

2.Desarrollo de módulos y modelos (I)

SISTEMAS DE GESTIÓN EMPRESARIAL

2ºDAM



Bibliografía

[Odoo 11 Development Essentials - Third Edition](#)

ISBN-13: 978-1788477796

[Odoo 11 Development Cookbook](#)

ISBN-13: 978-1788471817

[Working with Odoo 11 - Third Edition: Configure, manage, and customize your Odoo system](#)

ISBN-13: 978-1788476959

Introducción

- Los módulos de Odoo pueden agregar una nueva lógica comercial a un sistema Odoo o modificar y ampliar la lógica comercial existentes.
- Cualquier extensión en Odoo ya sea en servidor o en cliente son empaquetado como módulos.
- Los módulos pueden ser cargados opcionalmente en la base de datos de Odoo.
- Ejemplos de módulos: Agregar reglas contables de un país concreto, visualización en tiempo real de una flota de vehículos, etc.

Composición de un módulo

Un módulo de Odoo puede contener varios elementos como:

- Objetos de negocio: También conocidas como modelos. Cada modelo se define mediante una clase Python con una sintaxis específica y se desarrolla en un módulo Python (un fichero).
- Vistas de objetos: Define la visualización de la interfaz de usuario. Estas vistas son descritas utilizando ficheros XML que son incorporados al módulo Odoo y cuya instalación provoca que se almacenen en la base de datos.
- Archivos de información: Archivos XML y/o CSV que declaran los metadatos del modelo. Entre ellos se incluyen informes, datos de configuración, reglas de seguridad, parametrización de módulos, datos de demostración, etc.
- Controladores web que posibilitan el manejo de solicitudes de navegadores web.
- Datos web estáticos como imágenes, archivos CSS o ficheros JS utilizados para la interfaz de usuario.

Estructura de un módulo

- Cada módulo es un directorio que se encuentra dentro de la carpeta addons de Odoo. Los directorios que almacenan módulos Odoo se deben especificar mediante la opción --addons-path.
- Un módulo de Odoo se declara por su fichero de manifiesto (`__manifest__.py`).
- También es un paquete de Python ya que incluye un fichero (`__init__.py`) que contiene instrucciones de importaciones de varios archivos de Python requeridos para el mismo.

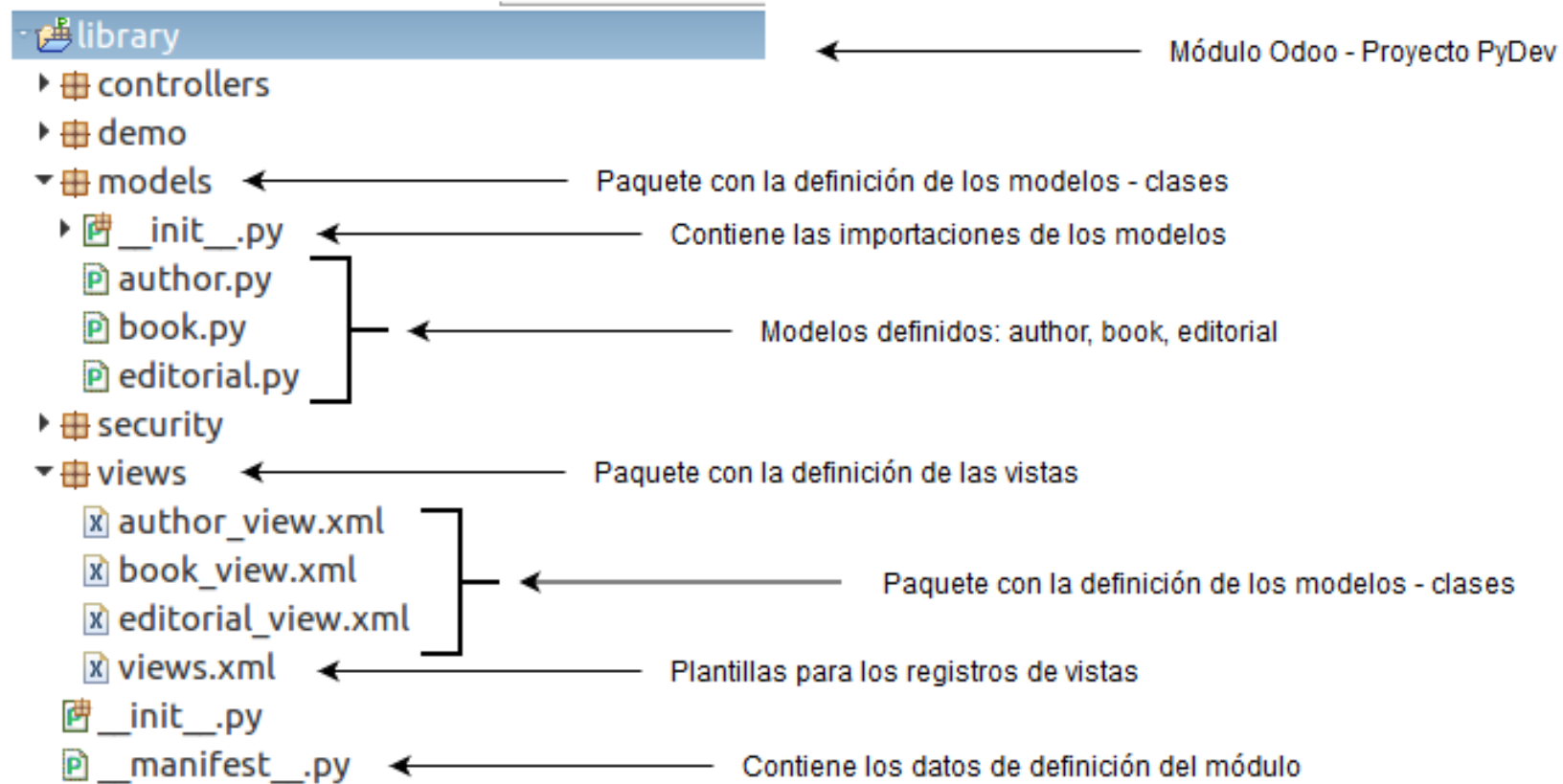
Estructura de un módulo

- Odoo proporciona una herramienta desde la línea de comandos que nos permite generar la estructura o esqueleto de un módulo Odoo.
- Para crear un módulo Odoo nuevo podemos utilizar el parámetro scaffold del ejecutable odoo.

```
odoo scaffold modulo_app addons
```

- Siendo modulo_app el nombre del nuevo módulo o app de Odoo
- El directorio addons es el directorio donde se almacenará la estructura de directorios del nuevo módulo de Odoo.

Estructura de un módulo



Fichero `__manifest__.py`

- Es el archivo de manifiesto del módulo Odoo y contiene información básica para su identificación, configuración y dependencias.
- Debe contener un diccionario con los pares clave/valor que se definen en Odoo.

Fichero `__manifest__.py`

Las claves de este fichero son:

- *name (Obligatorio)*: Cadena con el título del módulo.
- *version*: Versión del módulo. Por defecto, es la 1.0.
- *description*: Texto largo con la descripción de las características.
- *autor*: Nombre del autor o autores.
- *website*: Sitio web con una posible documentación del módulo Odoo.
- *license*: Licencia de distribución del módulo. GPL-2, GPL-3, AGPL-3, LGPL-3...
- *category*: Categoría que es utilizada para el filtrado de módulos en la sección “Apps” de Odoo. Las categorías disponibles se encuentra en: https://github.com/odoo/odoo/blob/13.0/odoo/addons/base/data/ir_module_category_data.xml
- *depends*: Lista de módulos dependientes del módulo Odoo. Se instalarán automáticamente antes que el propio módulo.

Fichero `__manifest__.py`

Las claves de este fichero son:

- *data*: Ficheros XML/CSV que deben ser cargados junto con la instalación del módulo Odoo. Ejemplos: reglas de seguridad, interfaces de usuario, etc.
- *demo*: Fichero/s XML cargados para el modo de demostración.
- *auto_install*: Valor booleano que instalará automáticamente el módulo Odoo si sus dependencias están instaladas previamente.
- *external_dependencies*: Diccionario que contiene dependencias de Python o de ficheros binarios.
- *application*: Valor booleano que indica si una aplicación es completa (true) o si es un módulo técnico que proporciona alguna funcionalidad adicional a un módulo ya existente (false).
- *css*: Archivos CSS con reglas personalizadas que deben importarse. Estos archivos deben almacenarse en `static/src/css` dentro del módulo.
- *images*: Archivos de imagen necesarios para el módulo.

Fichero `__manifest__.py`

Las claves de este fichero son:

- *summary*: Subtítulo del módulo.
- *installable*. Valor booleano que indica si un usuario puede instalar el módulo desde la interfaz web de Odoo. El valor por defecto es true.
- *maintainer*. Persona encargada del mantenimiento del módulo.
- *active*: Valor booleano que indica si el módulo se puede utilizar o no. Generalmente, esta opción se utiliza para archivar módulos obsoletos.

Fichero __manifest__.py

```
# -*- coding: utf-8 -*-
{
    'name': "library",
    'summary' : "Biblioteca de la UPO",
    'description': """
        Biblioteca de la UPO. Aplicación para su gestión.
    """,
    'category' : 'education',
    'website' : "http://biblioteca.upo.es",
    'author': "Carlos Rodríguez",

    # any module necessary for this one to work correctly
    'depends': ['base'],
    'data' : [ "author_view.xml", "book_view.xml", "editorial_view.xml"],
    'application': True,
}
```

Fichero `__init__.py`

- Contiene la importación de los modelos definidos en el módulo.
- Si un modelo no se incluye aquí es ignorado y por tanto no se crea su correspondiente tabla en la base de datos.

```
# -*- coding: utf-8 -*-  
  
from . import author  
from . import book  
from . import editorial
```

Fichero
`models/__init__.py`

Fichero controllers.py

- Fichero que contiene los diferentes enrutadores (routes) que afectan al módulo.
- Un enrutador es un componente encargado de organizar los diferentes estados de una aplicación, es decir, los estados que se producen en los cambios entre diferentes vistas.
- Ejemplo: El enrutador hará que se muestre un inicio de sesión inicial, y cuando el inicio de sesión sea exitoso, realizará la transición a la pantalla de bienvenida del usuario.

Fichero models.py

- Fichero que ofrece la posibilidad de almacenar todas las clases modelo utilizadas en un módulo de Odoo.
- La incorporación de todos los modelos en un único archivo ahorrará la modificación del fichero `__init__.py` del módulo.
- Sin embargo, existe la posibilidad de eliminar este fichero y crear un modelo por cada fichero Python. Esta manera de implementación hará que se deba modificar el fichero `__init__.py`

Fichero security/ir.model.access.csv

- Fichero CSV que almacena los parámetros de seguridad de acceso del módulo Odoo. Este archivo es utilizado para dar acceso a un módulo construido por nosotros a todos los usuarios de Odoo.
 - *id*: Identificador único para el permiso de seguridad.
 - *name*: Nombre único para el permiso de seguridad.
 - *model_id*: Nombre del modelo (clase) que quieres aplicarle los permisos.
 - *group_id*: Nombre del grupo de permisos asociado. Por defecto se dejará vacío.
 - *perm_read*: Valor booleano asociado al permiso de lectura del módulo a todos los usuarios.
 - *perm_write*: Valor booleano asociado al permiso de escritura a todos los usuarios.
 - *perm_create*: Valor booleano asociado al permiso de creación del módulo a todos los usuarios.
 - *perm_unlink*: Valor booleano asociado al permiso de desvinculación del módulo para todos los usuarios.

Fichero security/ir.model.access.csv

```
id,name,model_id/id,group_id/id,perm_read,perm_write,perm_create,perm_unlink  
access_oa,openacademy.openacademy,model_openacademy_openacademy,,1,0,0,0
```

Fichero templates.xml

- Fichero que ofrece la posibilidad de almacenar todas las interfaces de usuario en un único XML.
- La incorporación de todos las interfaces (templates) en un único XML ahorrará la modificación de la clave 'data' del fichero `__manifest__.py` del módulo.
- Sin embargo, existe la posibilidad de eliminar este fichero y crear una interfaz de usuario por cada fichero XML. Esta manera de implementación hará que se deba modificar el fichero `__manifest__.py`